

计算机组成原理实验报告

姓名: 林晖鹏 学号: 2312966 专业: 计算机科学与技术

目录

1 实验目的	2
2 实验要求	2
3 实验原理	2
3.1 五级流水线的五个阶段	2
3.2 流水线中的冒险	3
3.3 Verilog 实现要点	3
4 实验过程	4
4.1 配置 IP 核	4
4.2 现有问题分析	6
4.2.1 控制冒险	6
4.2.2 数据冒险	8
4.3 指令扩展	9
4.3.1 运算指令扩展-按位与非	9
4.3.2 转移指令扩展-相等跳转并链接	11
4.3.3 乘法指令扩展-无符号乘法	11
4.3.4 访存指令扩展-半字加载、半字储存	13
5 实验结果	14
6 实验总结	16
6.1 心得体会	17

报告摘要

关键字:

1 实验目的

- 在多周期 CPU 实验完成的前提下, 深入理解 CPU 流水线的概念。
- 熟悉并掌握流水线 CPU 的原理和设计。
- 最终检验运用 verilog 语言进行电路设计的能力。
- 通过亲自设计实现静态 5 级流水线 CPU, 加深对计算机组成原理和体系结构理论知识的理解。
- 培养对 CPU 设计的兴趣, 加深对 CPU 现有架构的理解和深思。

2 实验要求

1. 针对现有五级流水线存在问题的分析, 不只是 bug, 包括指令相关、流水线冲突等各种能分析的问题进行分析;
2. 五级流水线指令运行时的 bug 修复;
3. 五级流水线指令扩展, 运算类指令扩展至少一条, 乘除类指令至少一条, 转移指令至少一条, 访存指令一读一写两条;
4. 实验报告中针对 bug 修复过程、指令添加过程进行关键说明, 并最终验证, 验证需要有波形图或实验箱照片, 并对波形图和实验箱照片进行分析解释。

3 实验原理

五级流水线 CPU 是基于 MIPS 指令集体系结构的一种高性能处理器设计方法, 通过将指令执行分为五个阶段 (取指、译码、执行、访存、写回), 实现指令的并行处理, 从而提高 CPU 的吞吐率。以下详细介绍五级流水线的原理及其关键机制。

3.1 五级流水线的五个阶段

五级流水线将指令执行过程分为以下五个阶段, 每个阶段在一个时钟周期内完成:

1. **取指 (IF)**: 从指令存储器中读取指令, 根据程序计数器 (PC) 的值获取指令内容, 并将 PC 值加 4 以准备下一条指令的取指。
2. **译码 (ID)**: 解析指令的操作码和操作数, 读取寄存器文件中的源寄存器值, 并生成控制信号。同时, 计算分支指令的目标地址。
3. **执行 (EXE)**: 执行算术或逻辑运算, 计算地址 (对于访存指令) 或分支条件。根据指令类型, ALU 完成加、减、移位等操作, 或判断分支是否成立。
4. **访存 (MEM)**: 对于访存指令 (如 `lw` 和 `sw`), 访问数据存储器进行读或写操作; 对于非访存指令, 此阶段通常为空操作 (NOP)。
5. **写回 (WB)**: 将运算结果或访存读取的数据写回寄存器文件。R 型指令 (如 `add`) 和加载指令 (如 `lw`) 在此阶段更新目标寄存器。

3.2 流水线中的冒险

五级流水线在提高性能的同时，也引入了以下潜在问题（冒险），需要通过硬件或软件机制解决：

- **数据冒险**：当后续指令依赖前一条指令尚未完成的结果时，可能读取错误数据。例如，指令 `addiu $1, $0, #1` 后紧跟 `sll $2, $1, #4`，译码阶段可能无法获取 `$1` 的正确值。解决方法包括前递（Forwarding）或插入流水线气泡（Stall）。
- **控制冒险**：分支指令（如 `bgez`）可能导致取指阶段获取错误的指令。例如，分支指令的目标地址在译码阶段才确定，而此时下一条指令可能已被错误取指。解决方法包括分支预测或延迟槽。
- **结构冒险**：多个指令同时竞争同一硬件资源（如存储器）。在 MIPS 架构中，通常通过分离指令存储器和数据存储器来缓解此问题。

3.3 Verilog 实现要点

在 Verilog 实现中，五级流水线通过模块化设计实现，每个阶段由独立的模块或 `always` 块控制，流水线寄存器用于在阶段间传递数据和控制信号。例如：

- **流水线寄存器**：如 IF/ID 寄存器存储取指的指令和 PC 值，ID/EX 寄存器存储译码后的控制信号和操作数。
- **控制单元**：根据指令的操作码生成控制信号（如 `RegWrite`、`MemRead`），控制 ALU、存储器和寄存器文件的操作。
- **冒险处理**：通过前递单元检测寄存器依赖，修改 `ID_valid` 逻辑处理分支指令的控制冒险。

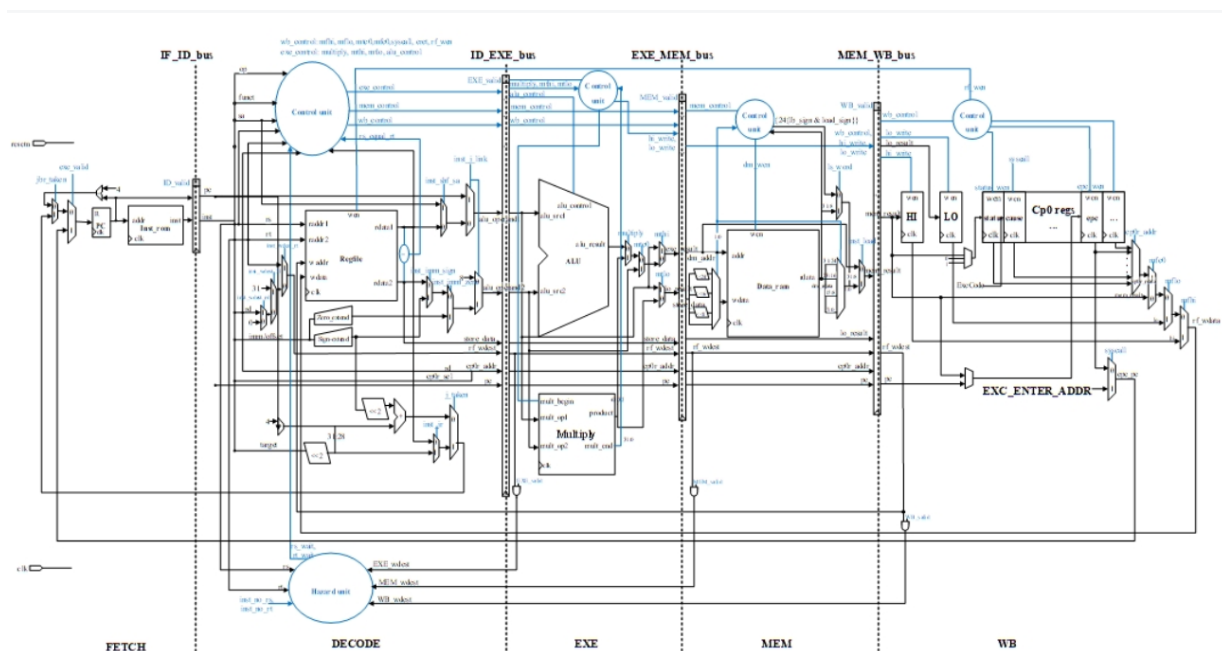


图 3.1: 流水线原理图

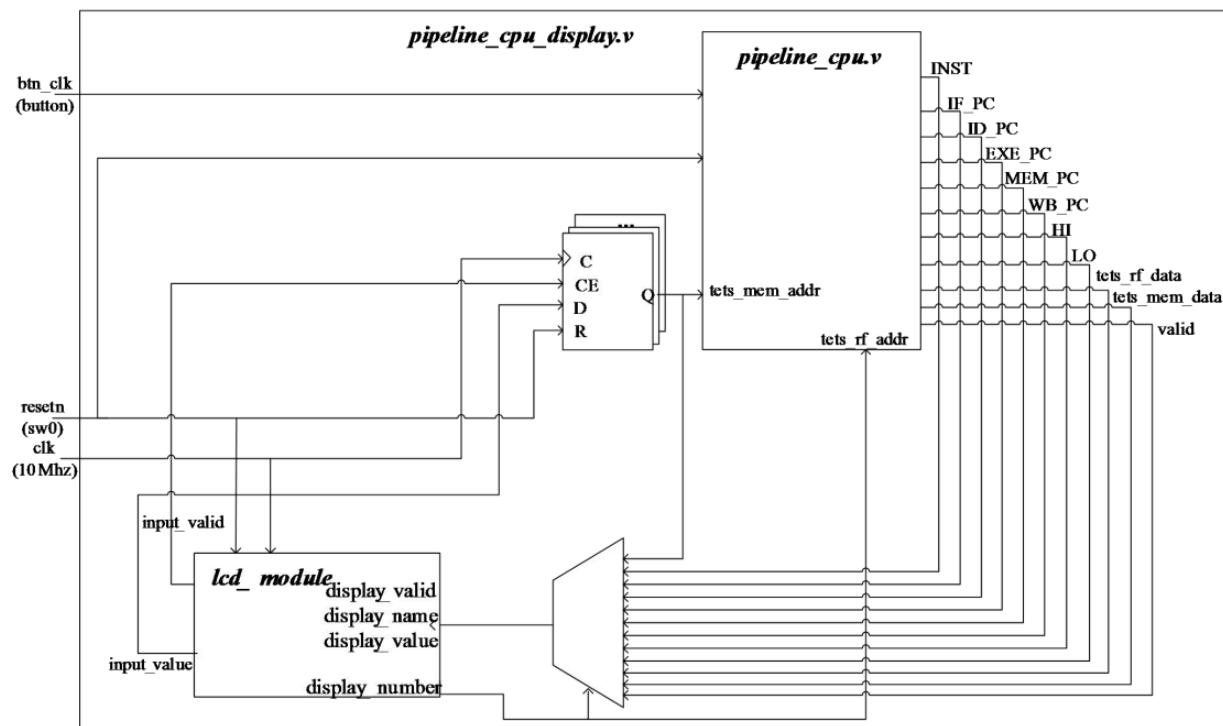
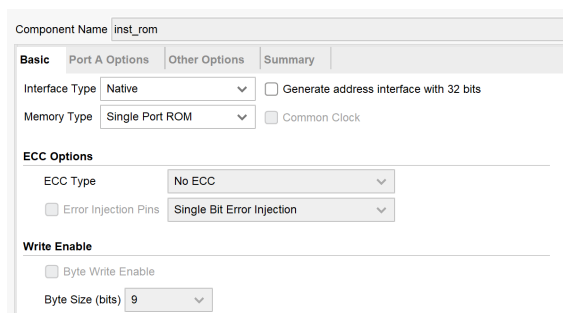


图 3.2: 五级流水线顶层模块框架图

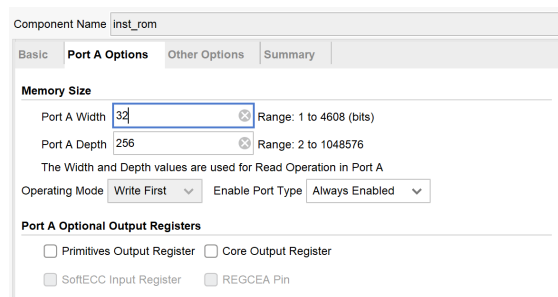
4 实验过程

4.1 配置 IP 核

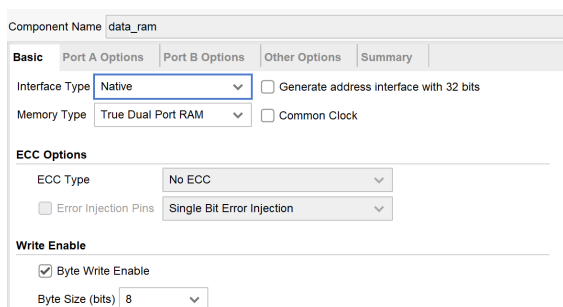
为实现五级流水线 CPU 的指令存储器和数据存储器，使用 Xilinx Vivado 的 Block Memory Generator 创建 IP 核。以下通过五张截图展示 IP 核配置向导的关键步骤，确保存储器满足流水线设计的时序要求。



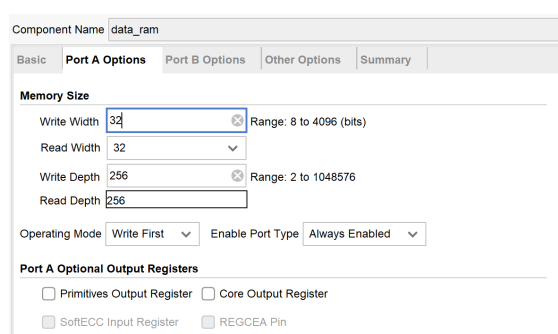
(a) 选择 Block Memory Generator IP 核



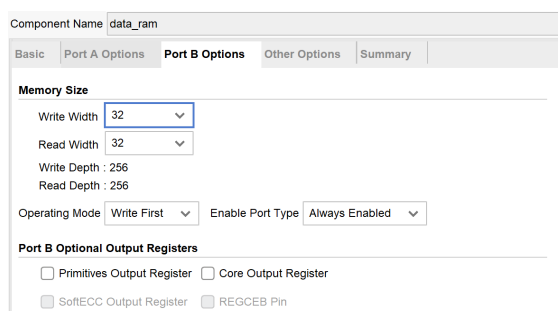
(b) 配置存储器类型为单端口 ROM（用于指令存储器）



(c) 设置存储器大小和数据宽度（32 位，匹配 MIPS 指令）



(d) 配置读写端口参数（同步读，单周期延迟）



(e) 检查输出寄存器选项，确保不勾选 Primitives Output Register

图 4.3: IP 核配置向导的关键步骤，用于创建指令存储器

在配置 IP 核时，需特别注意不勾选 **Primitives Output Register** 选项。

- **功能说明：** Primitives Output Register 选项控制是否在存储器的原始输出端口后添加一个 D 触发器（寄存器）。若勾选，存储器的输出数据会经过一级寄存器缓存，导致输出延迟增加一个时钟周期。同步存储器（如 Block RAM）本身已有单周期读取延迟（从地址输入到数据输出需一个时钟周期），因此勾选此选项会使总延迟达到两个时钟周期。
- **对取指阶段的影响：** 在五级流水线 CPU 的取指（IF）阶段，指令存储器根据程序计数器（PC）读取指令，设计假设指令在一个时钟周期内可用（单周期锁存）。若输出延迟为两周期，指令无法在预期时间内到达 IF/ID 流水线寄存器，可能导致译码阶段处理错误的指令或时序错乱。

调整完之后，流水线可以成功运行：

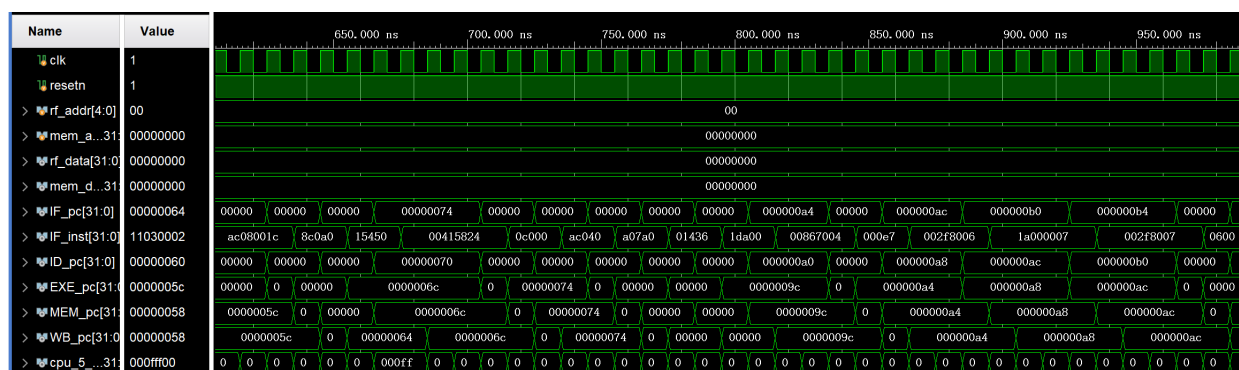


图 4.4: 流水线运行图

4.2 现有问题分析

4.2.1 控制冒险

控制冒险的原理 控制冒险是五级流水线 CPU 中的一种常见问题，发生在分支指令（如 `bgez`、`beq`、`jalr`）执行时。由于分支指令的目标地址通常在译码（ID）或执行（EX）阶段才能确定，而取指（IF）阶段会继续获取后续顺序指令（PC+4），可能导致错误指令被取入流水线，影响程序的正确执行。

在五级流水线（取指、译码、执行、访存、写回）中，分支指令的处理流程如下：

- 在 **取指（IF）阶段**，根据程序计数器（PC）从指令存储器获取指令。例如，地址 0x48 处的 `bgez $25, #14` 指令被取指，同时 PC 更新为 0x4C（PC+4）。
- 在 **译码（ID）阶段**，解析指令并计算分支目标地址（如 0x84）。此时，IF 阶段已取指下一条指令（0x4C）。
- 若分支条件成立（如 \$25 满足 `bgez` 条件），应跳转到目标地址 0x84。但由于流水线的并行性，0x4C 的指令已被错误取入流水线，可能继续进入译码和执行阶段，导致错误的结果。

控制冒险的主要影响是错误指令的执行，需通过以下方法解决：

- 插入流水线气泡**：暂停 IF 和 ID 阶段，直到分支目标地址确定，防止错误指令传播。
- 分支预测**：预测分支是否跳转，错误预测时冲刷流水线（Flush），清除错误取指的指令。
- 延迟槽**：MIPS 架构中，分支指令后的延迟槽指令总是执行，需由编译器优化。
- 动态控制**：通过控制信号（如 `ID_valid`）动态调整流水线，确保仅当 PC 匹配分支目标时才允许译码。

以下通过时序图展示分支冒险的发生过程，突出错误取指的问题。

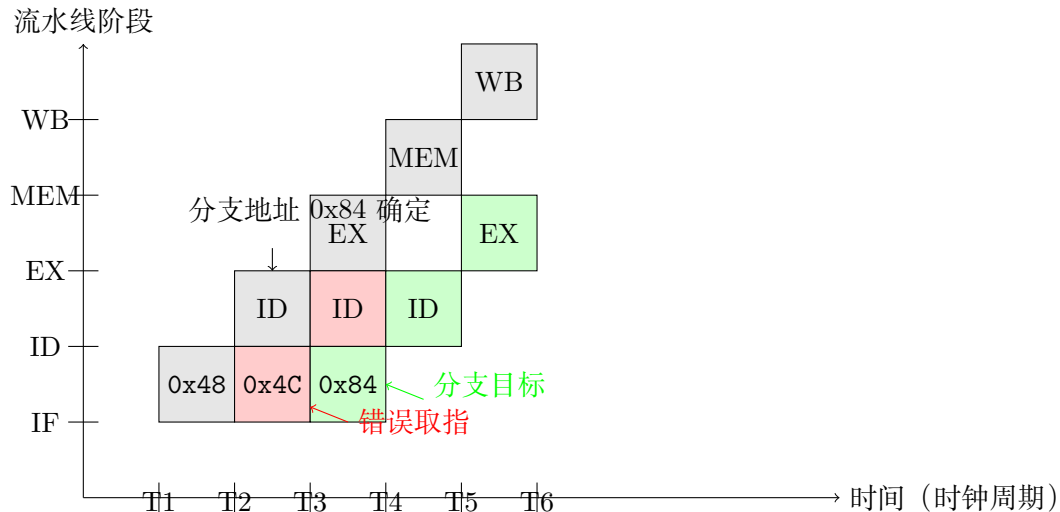


图 4.5: 分支冒险时序图, 展示 bgez 指令导致的错误取指

案例 在实验中, 发现地址 0x48 处的跳转指令 bgez \$25, #14 应跳转到地址 0x84。然而, 由于流水线的取指 (IF) 和译码 (ID) 阶段同时进行, 下一条指令 0x4C 在分支目标地址解析前已被取指, 导致 0x4C 错误地进入流水线并继续执行。

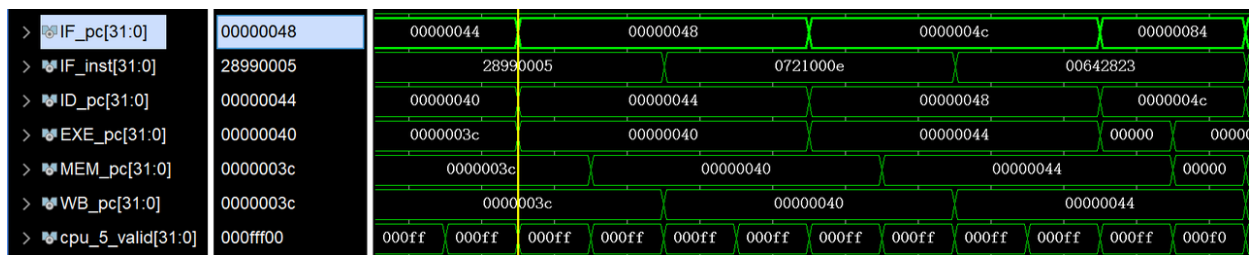


图 4.6: 分支冒险波形图

为解决此问题, 在 ID_valid 逻辑中添加了条件判断, 确保当 ID 阶段识别到跳转指令时, 仅在 IF 阶段取指的 PC 等于跳转目标地址 (0x84) 时, 才设置 ID_valid 为有效, 从而允许目标指令的译码。修改后的 Verilog 代码如下:

```

1 //ID_valid
2 always @(posedge clk)
3 begin
4     if (!resetn || cancel)
5     begin
6         ID_valid <= 1'b0;
7     end
8     else if (ID_allow_in)
9     begin
10        // 只有在非跳转指令或者IF_pc已经等于跳转目标地址时, 才设置ID_valid
11        if (~jbr_bus[32] | IF_pc == jbr_bus[31:0])
12        begin

```

```

13     ID_valid <= IF_over;
14 end
15 else
16 begin
17     ID_valid <= 1'b0; // 如果是跳转指令但目标地址尚未准备好, 保持ID_valid为0
18 end
19
20 end
21 end

```

此修改确保流水线在分支指令处理时暂停译码, 直到正确取到目标指令 (如 0x84), 避免执行错误的 0x4C 指令。下面波形图展示了分支冒险及其修复效果。

此时, 即使 0x4C 进入了流水线, 但是并没有真正执行, 可以看到 \$5 是一直没有赋值的, 说明我们成功暂停了额外取码的指令执行。

但是修复了之后这个指令集跑不通了, 所以这里的修改部分还是留到体系结构吧呜呜呜呜。

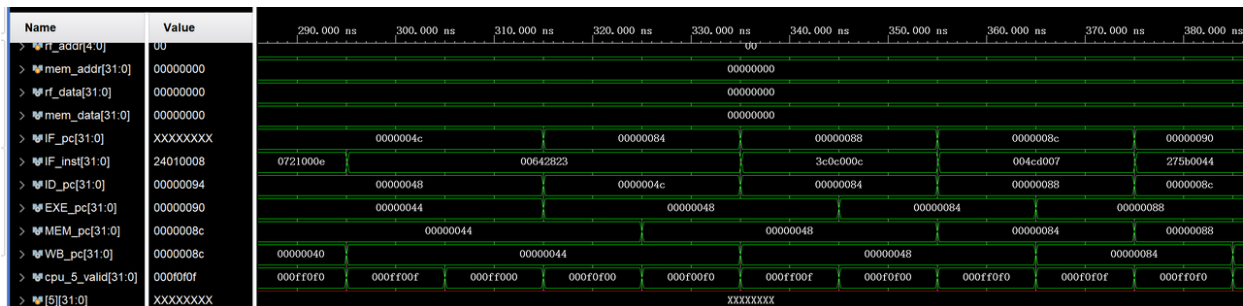


图 4.7: 修复分支冒险 bug 后波形图

4.2.2 数据冒险

数据冒险是五级流水线 CPU 中的一种常见问题, 发生在后续指令依赖前一条指令的结果 (寄存器值或内存数据), 但前指令的结果尚未写回寄存器文件或内存时。在五级流水线 (取指 IF、译码 ID、执行 EX、访存 MEM、写回 WB) 中, 指令并行执行可能导致数据依赖冲突。

数据冒险的主要类型为 **RAW (Read After Write)**, 即后续指令读取前指令尚未写回的寄存器值。例如, 指令 A 在 EX 阶段计算结果, 需在 WB 阶段写回寄存器 \$1, 而指令 B 在 ID 阶段读取 \$1, 可能获取旧值或未定义值 (XXXX)。数据冒险的发生原因如下:

- **流水线并行性:** IF 和 ID 阶段提前执行, 而前指令的结果在 MEM 或 WB 阶段才可用。
- **寄存器读写时序:** 译码阶段读取寄存器值早于写回阶段, 导致数据依赖冲突。

数据冒险的解决方法包括:

- **前递:** 从 EX 或 MEM 阶段直接将结果传递到 ID 或 EX 阶段。例如, 将 ALU 结果直接前递给后续指令的 ALU 输入。
- **流水线暂停:** 暂停 IF 和 ID 阶段, 直到依赖数据写回寄存器文件。
- **编译器优化:** 通过指令重排减少数据依赖。

流水线暂停 而在本实验的原本代码中, 所有操作数的读取都在译码 ID 阶段完成。当遇到数据冒险的时候, 本代码采用了流水线暂停的方式避免这种风险。

在 ID 阶段, 代码中设置了一个 wait 信号, decode 模块接收来此其他模块的要写回寄存器堆的目标地址号, 然后和当前译码时要取得的地址做对比, 如果相同, 就停止等待, 让 ID_over 信号一直为 0, 直到其他模块写完了, 其实就是等到 WB 开始 (也就是结束, 因为 WB 阶段能在一拍内结束), 才结束现在的 ID, 进入 EXE 阶段, 同时允许下一个指令进入 ID 阶段。

这样其实就是一旦出现数据依赖, 就把流水线暂停, 直到数据依赖消失, 实现核心代码如下:

```

1  wire rs_wait;
2  wire rt_wait;
3  assign rs_wait = ~inst_no_rs & (rs!=5'd0)
4                & ( (rs==EXE_wdest) | (rs==MEM_wdest) | (rs==WB_wdest) );
5  assign rt_wait = ~inst_no_rt & (rt!=5'd0)
6                & ( (rt==EXE_wdest) | (rt==MEM_wdest) | (rt==WB_wdest) );
7  assign ID_over = ID_valid & ~rs_wait & ~rt_wait & (~inst_jbr | IF_over) ;

```

数据前递 显而易见, 这样让流水线暂停会让性能大大降低。写回的结果往往在 EXE 执行阶段或者 MEM 访存阶段就已经得到, 如果我们能将这些阶段的输出结果直接前递到下一条指令的 ID 阶段, 这样就能大大提升效率。

我们可以在 EXE 和 MEM 阶段设置输出端口 (如 EX_result、MEM_result), 记录 ALU 或访存结果。

然后前递单元比较当前指令的源寄存器 (rs、rt) 与前指令的目标寄存器 (rd 或 rt), 若匹配, 则通过多路选择器 (MUX) 将 EXE/MEM 结果前递到 ID/EX 阶段的 ALU 输入。

4.3 指令扩展

本节对指令进行扩展:

4.3.1 运算指令扩展-按位与非

1. 修改 alu.v

```

1  //alu控制信号位扩展
2  input [12:0] alu_control, // ALU控制信号
3  ...
4  //添加控制信号
5  wire alu_nand; //按位与非
6  assign alu_nand = alu_control[12];
7  ...
8  //计算结果
9  wire [31:0] nand_result;
10 assign nand_result = ~(alu_src1 & alu_src2);
11 ...
12 //添加结果输出

```

```

13     assign alu_result = (alu_add|alu_sub) ? add_sub_result[31:0] :
14         alu_slt      ? slt_result :
15         alu_sltu     ? sltu_result :
16         alu_and      ? and_result :
17         alu_nor      ? nor_result :
18         alu_or       ? or_result :
19         alu_xor      ? xor_result :
20         alu_sll      ? sll_result :
21         alu_srl      ? srl_result :
22         alu_sra      ? sra_result :
23         alu_lui      ? lui_result :
24         alu_nand     ? nand_result :
25         32'd0;

```

2. 修改 decode.v

```

1     //添加指令信号列表
2     wire inst_NAND;
3     assign inst_NAND = op_zero & sa_zero & (funct == 6'b110001);
4     //添加信号
5     wire inst_nand;
6     assign inst_nand = inst_NAND;
7     //修改写回rd使能端信号
8     assign inst_wdest_rd = inst_ADDU | inst_SUBU | inst_SLT | inst_SLTU
9         | inst_JALR | inst_AND | inst_NOR | inst_OR | inst_NAND
10        | inst_XOR | inst_SLL | inst_SLLV | inst_SRA
11        | inst_SRAV | inst_SRL | inst_SRLV
12        | inst_MFHI | inst_MFLO;
13    //修改控制信号
14    assign alu_control = {inst_nand,
15        inst_add,      // ALU操作码, 独热编码
16        inst_sub,
17        inst_slt,
18        inst_sltu,
19        inst_and,
20        inst_nor,
21        inst_or,
22        inst_xor,
23        inst_sll,
24        inst_srl,
25        inst_sra,
26        inst_lui};

```

3. 修改其他文件剩下的修改都是一些变量（比如总线）的长度需要修改，这里不赘述。

4.3.2 转移指令扩展-相等跳转并链接

1. 修改 decode.v

```

1 //增加信号
2 wire inst_BGEZAL;
3 assign inst_BGEZAL = (op == 6'b011111);
4 //
5 assign inst_j_link = inst_JAL | inst_JALR | inst_BGEZAL;
6 assign inst_jbr = inst_J | inst_JAL | inst_jr
7                 | inst_BEQ | inst_BNE | inst_BGEZ
8                 | inst_BGTZ | inst_BLEZ | inst_BLTZ
9                 | inst_BGEZAL;
10 //
11 assign inst_wdest_31 = inst_JAL | inst_BGEZAL;
12 //
13 assign br_taken = inst_BEQ & rs_equql_rt // 相等跳转
14                 | inst_BNE & ~rs_equql_rt // 不等跳转
15                 | inst_BGEZ & ~rs_ltz      // 大于等于0跳转
16                 | inst_BGTZ & ~rs_ltz & ~rs_ez // 大于0跳转
17                 | inst_BLEZ & (rs_ltz | rs_ez) // 小于等于0跳转
18                 | inst_BLTZ & rs_ltz      // 小于0跳转
19                 | inst_BGEZAL & ~rs_ltz;

```

2. BGEZAL 在取指和译码阶段的跳转条件判断及 PC 生成逻辑与 BLTZ 和 BGEZ 相同，在执行、访存、写回阶段的 PC+8 计算及写入 \$31 逻辑与 JAL 一致。所以其他地方无需修改，

4.3.3 乘法指令扩展-无符号乘法

1. 修改 decode.v

```

1 //添加信号
2 wire inst_MULTU;
3 assign inst_MULTU = op_zero & (rd==5'd0) & sa_zero & (funct == 6'b110010);
4 wire unsigned_multiply;
5 assign unsigned_multiply = inst_MULTU;
6 //信号加入到总线中
7 assign ID_EXE_bus = {unsigned_multiply,multiply,mthi,mtlo,
8                     //EXE需用的信息,新增
9                     alu_control,alu_operand1,alu_operand2,//EXE需用的信息
10                    mem_control,store_data, //MEM需用的信号
11                    mfhi,mflo, //WB需用的信号,新增

```

```
11         mtc0,mfc0,cp0r_addr,syscall,eret, //WB需用的信号,新增
12         rf_wen, rf_wdest,                //WB需用的信号
13         pc};                             //PC值
```

2. 修改 exe.v

```
1 //添加信号, 并从总线添加读取
2 wire unsigned_multiply;
3 //修改信号控制
4 assign mult_begin = (multiply|unsigned_multiply) & EXE_valid;
5 //添加符号控制
6 multiply multiply_module (
7     .clk      (clk      ),
8     .mult_begin(mult_begin ),
9     .mult_op1  (alu_operand1),
10    .mult_op2  (alu_operand2),
11    .product   (product  ),
12    .mult_end  (mult_end ),
13    .sign(~unsigned_multiply)
14 );
15 //结束信号修改
16 assign EXE_over = EXE_valid & ((~multiply&~unsigned_multiply) | mult_end);
17 //修改结果写入条件
18     assign exe_result = mthi ? alu_operand1 :
19                     mtc0    ? alu_operand2 :
20                     (multiply|unsigned_multiply) ? product[63:32] : alu_result;
21 assign lo_result = mtlo ? alu_operand1 : product[31:0];
22 assign hi_write = multiply| unsigned_multiply | mthi;
23 assign lo_write = multiply| unsigned_multiply | mtlo;
```

3. 修改 multiply.v

```
1 //模块定义中添加输入
2 module multiply(                // 乘法器
3     input      clk,            // 时钟
4     input      mult_begin,     // 乘法开始信号
5     input [31:0] mult_op1,     // 乘法源操作数1
6     input [31:0] mult_op2,     // 乘法源操作数2
7     output [63:0] product,     // 乘积
8     output      mult_end,      // 乘法结束信号
9     input      sign
10 );
```

```
11 assign op1_sign = sign ? mult_op1[31] : 1'b0;
12 assign op2_sign = sign ? mult_op2[31] : 1'b0;
13 assign op1_absolute = (sign && op1_sign) ? (~mult_op1 + 1) : mult_op1;
14 assign op2_absolute = (sign && op2_sign) ? (~mult_op2 + 1) : mult_op2;
15 //sign为1的时候说明有符号，为0的时候说明无符号，直接修改最终符号位计算
16 product_sign <= sign ? op1_sign ^ op2_sign : 1'b0;
```

4. 修改其他文件剩下的修改都是一些变量（比如总线）的长度需要修改，这里不赘述。

4.3.4 访存指令扩展-半字加载、半字储存

1. 修改 decode.v

```
1 //增加控制指令
2 wire inst_LH,inst_SH;
3 assign inst_LH = (op == 6'b011110);
4 assign inst_SH = (op == 6'b011101);
5 //增加指令信号
6 assign inst_load = inst_LW | inst_LB | inst_LBU | inst_LH; // load指令
7 assign inst_store = inst_SW | inst_SB | inst_SH; // store指令
8 //增加访存要用到的信息
9 wire lh_sign;
10 wire ls_halfword;
11 assign lh_sign = inst_LH;
12 assign ls_halfword = inst_LH | inst_SH;
13 assign mem_control = {inst_load,
14                       inst_store,
15                       ls_word,
16                       ls_halfword,
17                       lb_sign|lh_sign };
18 //修改变量长度...
```

2. 修改 mem.v

```
1 //增加信号读取
2 wire ls_halfword;
3 assign {inst_load,inst_store,ls_word,ls_halfword,lb_sign} = mem_control;
4
5 //添加SH写使能判断分支
6 else if (ls_halfword)//SH
7     begin
8         case (dm_addr[1:0])
9             2'b00: dm_wen <= 4'b0011; // 地址对齐，写入低 2 字节
```

```

10      2'b10: dm_wen <= 4'b1100; // 地址偏移 2 字节, 写入高 2 字节
11      default: dm_wen <= 4'b0000; // 非对齐地址 (如 01, 11), 禁用写入
12      endcase
13      end
14      //添加SH的写操作
15      always @ (*)
16      begin
17          case (dm_addr[1:0])
18              2'b00 : dm_wdata <= store_data;
19              2'b01 : dm_wdata <= {16'd0, store_data[7:0], 8'd0};
20              2'b10 : dm_wdata <= ls_halfword ? {store_data[15:0],16'd0} : {8'd0,
                store_data[7:0], 16'd0};
21              2'b11 : dm_wdata <= {store_data[7:0], 24'd0};
22              default : dm_wdata <= store_data;
23          endcase
24      end
25      //添加LH结果计算
26      wire      load_half_sign;
27      wire [31:0]load_result2;
28      assign load_half_sign = (dm_addr[1:0]==2'd0) ? dm_rdata[15] : dm_rdata[31];
29      assign load_result2[15:0] = (dm_addr[1:0]==2'd0) ? dm_rdata[15:0] :
        dm_rdata[31:16];
30      assign load_result2[31:16]= {16{lb_sign & load_half_sign}};
31      //修改结果判断
32      assign mem_result = inst_load ? (ls_halfword ? load_result2 : load_result)
        : exe_result;

```

3. 修改其他文件剩下的修改都是一些变量 (比如总线) 的长度需要修改, 这里不赘述。

5 实验结果

为了防止加指令之后跳转偏移量改变, 我们这里加到倒数第二条指令 (也就是 0x1b0) 后面。

- 访存指令验证

我们在后面插入两条指令:

```

1      74F00001 // (SH $16, ,#1($7))
2      78FE0001 // (LH $30, ,#1($7))

```

先是将寄存器 \$16 的值加载到内存中, 然后再把低 16 位读取到寄存器 \$30 处, 由于 \$16 为 ffffffff, 而 LH 是有符号位扩展, 所以此时 \$30 寄存器里面的值为 ffffffff, 如图中所示:

图 5.8: 访存指令验证

- 与非指令验证

我们在后面插入一条与非指令:

```
1      03FFE031 // (nand $28, $31, $31)
```

将寄存器 \$31 的值与自己进行与非计算, 得到结果存到寄存器 \$28 中, 从图中可以看到, 28 的值被改变, 而且是正确结果。

图 5.9: 与非指令验证

- 跳转指令验证

我们在后面插入三条指令:

```
1      0x1b4 7FE10002 (bgezal $31, #2, [$31] =0000_01bcH)
2      //这两条是与非指令
3      0x1b8 03FFE031
4      0x1bc 03FFF031
```

比较寄存器 \$31 的值是否大于等于 0, 这里 0x84 确实大于等于 0, 所以进行跳转, 跳转两个字节后的地址, 也就是图中所示的 0x1c0 处, 然后把地址链接到 \$31 寄存器处, 可以看到图中寄存器值也发生了改变。所以指令执行成功。

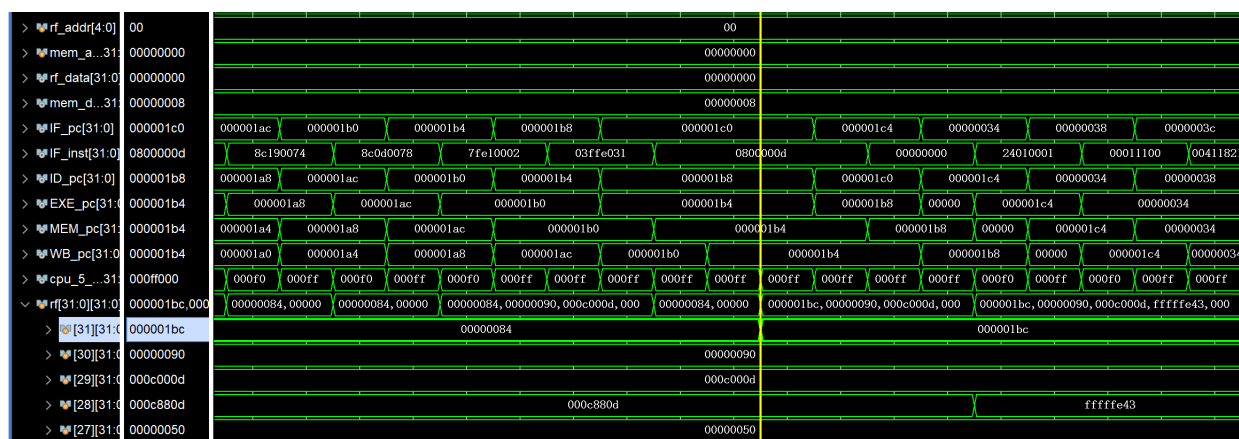


图 5.10: 跳转指令验证

• 乘法指令验证

```

1      0x1b4 03FF0032 (multu $31,$31)
2      0x1b8 0000E812 (mflo $29)
3      0x1bc 0000F010 (mfhi $30)

```

乘法指令执行的时候, 结果并不存在某个寄存器中, 因为寄存器只有 32 位, 而是存在了特定的寄存器中, 用 mflo 和 mfhi 能把结果的高位和低位取出来。

这里我们在最后面加了一个 $\$31 * \31 的乘法, 然后取出低位和高位分别放到寄存器 \$29 和 \$30, 可以从图中看到, 结果正确。

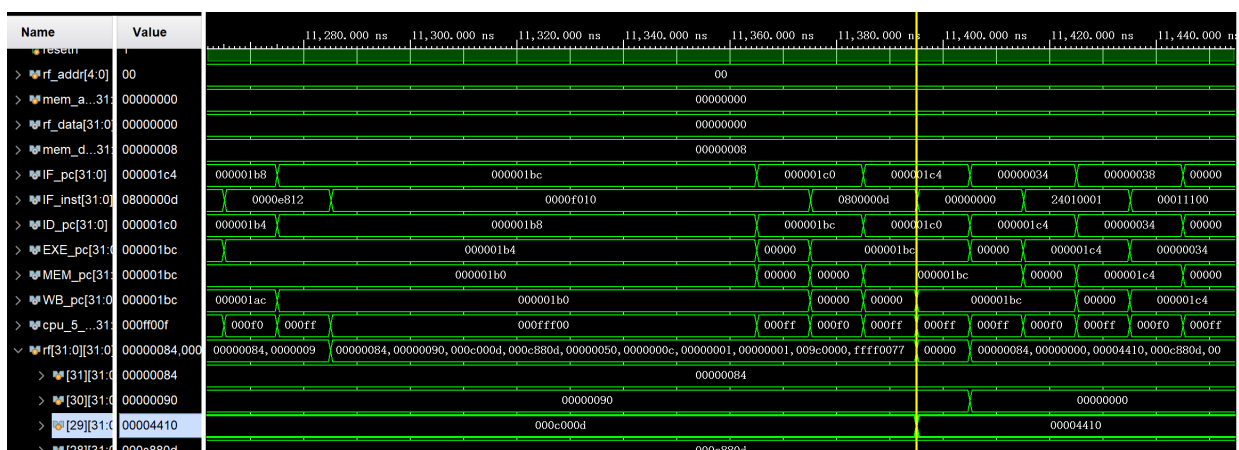


图 5.11: 无符号位乘法指令验证

6 实验总结

本次实验中, 我们通过分析调整 IP 核的设置, 成功让流水线跑起来。然后又对流水线存在的一些性能问题进行了分析: 数据冒险、控制冒险之类的, 并且对其进行了尝试, 但是最后老师说不需要改这个, 因为我改完之后发现指令集跑不了了...

然后实验的主体部分还是对各种指令的扩展, 虽然部分和之前很像, 但是还是多了几个之前没有接触过的类型, 在加上五级流水线的框架, 还是不简单的。

通过本实验，对五级流水线的架构，具体执行过程有了超级进一步的理解，相信以后体系结构上手能更快。

6.1 心得体会

本学期的机组实验就结束了，再次再次感谢董老师的帮助，无论是帮忙 debug 还是答疑，都超级超级耐心，引导学生而不是考验学生，从 verilog 课就看出来了，这老师不赖。